

✓ Estimacion de precio de alojamientos Airbnb en Barcelona

Script incluye imputación de variable Price, se anexa como evidencia de R2 similar al del modelo sin imputación

✓ 0. Configuración del entorno

Importamos todas las librerías necesarias que se van a usar a lo largo del notebook. Fijamos también una semilla aleatoria

```
# Instalar requirements con su versión
!pip install -r '/content/requirements.txt'
```

[Mostrar salida oculta](#)

```
# Importamos todas las librerías necesarias
import warnings
warnings.filterwarnings('ignore')
import pandas as pd
import numpy as np
import sys
import seaborn as sns
import matplotlib.pyplot as plt
import re
import patsy
import ast
import warnings
from pathlib import Path
from sklearn.model_selection import train_test_split, cross_val_score, RepeatedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, LogisticRegression, Ridge, Lasso
from sklearn import metrics
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score, mean_absolute_error, median_absolute_error, f1_score
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor, GradientBoostingRegressor
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.tree import DecisionTreeRegressor
from sklearn.neighbors import KNeighborsRegressor
from xgboost import XGBRegressor
from sklearn.model_selection import RandomizedSearchCV
import xgboost as xgb
import torch
import torch.nn as nn
from relativeImp import relativeImp
import matplotlib.pyplot as plt
from IPython.display import Image, display
from matplotlib import colormaps
from ydata_profiling import ProfileReport
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from scipy.stats import chi2_contingency
import geopandas as gpd
import pickle
import sklearn.impute as skl_imp
import shap
import lime
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Fijamos una semilla
SEED= 42

np.random.seed(SEED)
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x7f0a9a18d5b0>
```

```
# Funciones que utilizamos de ML y propias del TFM
execfile("/content/FuncionesTFM.py")
```

2. Comprensión de los datos

```
# Leer datos
df_marzo = pd.read_csv("/content/airbnb_marzo.csv")
df_marzo.info()
```

[Mostrar salida oculta](#)

✓ Número de filas y columnas y contenido

```
#eliminar las variables URLs
df_marzo = df_marzo.drop(columns=['listing_url', 'picture_url', 'host_url', 'host_profile_url', 'host_picture_url'])
```

```
# cambiar formato bool
variables_booleanas = ['host_is_superhost', 'host_has_profile_pic', 'host_identity_verified', 'has_availability']
df_marzo[variables_booleanas] = (
    df_marzo[variables_booleanas]
    .replace({"t": 1, "f": 0})
)
df_marzo[variables_booleanas] = df_marzo[variables_booleanas].astype("boolean")
```

```
#cambiar fechas
cols_fechas = ['last_scraped', 'price_quote_checkin_date', 'price_quote_checkout_date', 'calendar_last_scraped', 'first_review_date']
df_marzo[cols_fechas] = df_marzo[cols_fechas].apply(
    lambda col: pd.to_datetime(col, format="%Y-%m-%d", errors="coerce")
)
```

```
#cambiar categoricas
cols_categoricas = ['neighbourhood_cleansed', 'neighbourhood_group_cleansed', 'property_type', 'room_type']
df_marzo[cols_categoricas] = df_marzo[cols_categoricas].astype("category")
```

```
#cambiar string (hasta el momento, mientras trabajamos con este texto)
cols_string = ['amenities', 'source', 'name', 'host_name', 'description', 'host_location', 'host_about', 'bathrooms_text']
df_marzo[cols_string] = df_marzo[cols_string].astype("string")
```

```
#eliminar variables totalmente nulas
df_marzo = df_marzo.dropna(axis=1, how='all')
print(f"Dataset actualizado: {df_marzo.shape}")
```

Dataset actualizado: (16107, 73)

```
#convertimos en formato numero
print(df_marzo['price'].dropna().head(5).tolist())

df_marzo['price'] = df_marzo['price'].replace(['\\$', ','], '', regex=True).astype(float)

df_marzo['price'].describe()
```

[Mostrar salida oculta](#)

```
#añadimos columna de duracion estancia
df_marzo['duracion_estancia'] = (df_marzo['price_quote_checkout_date'] - df_marzo['price_quote_checkin_date']).dt.days
print(df_marzo['duracion_estancia'])
```

[Mostrar salida oculta](#)

```
df_marzo.nlargest(1, "price")[["duracion_estancia", "minimum_nights"]] # son 32 noches
```

[Mostrar salida oculta](#)

```
corr_noches = df_marzo[['duracion_estancia', 'minimum_nights']].corr()
corr_noches
#nos quedamos con minimum nights
```

[Mostrar salida oculta](#)

Pasos siguientes: [Generar código con corr_noches](#) [New interactive sheet](#)

```
# Eliminamos la variable creada, ya que lleva correlacion con minimum_nights
df_marzo = df_marzo.drop(columns=['duracion_estancia', 'price_quote_checkin_date', 'price_quote_checkout_date'])
```

✧ Familia variables price_quote

```
#eliminamos columnas redundantes de precio
df_marzo=df_marzo.drop(columns=['price_quote_total_price', 'price_quote_price_per_night', 'price_quote_raw'])
```

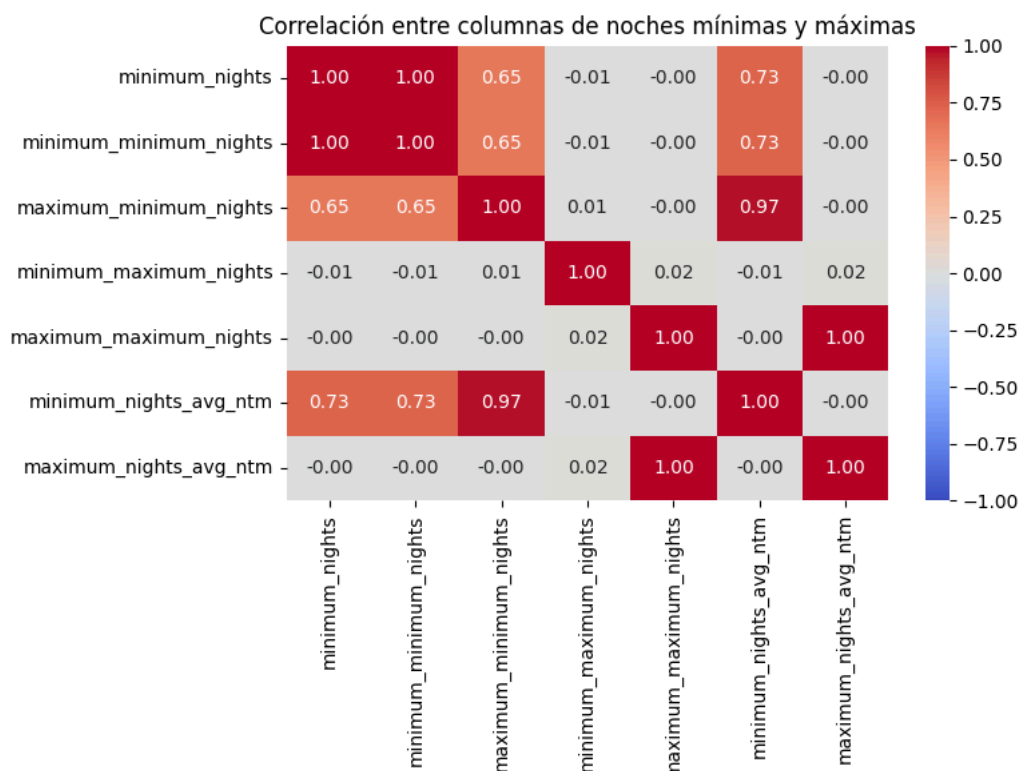
✧ Familia de columnas que comienzan con *minimum *maximum - noches mínimas / máximas

```
cols_noches = [ 'minimum_nights', 'minimum_minimum_nights', 'maximum_minimum_nights',
                'minimum_maximum_nights', 'maximum_maximum_nights',
                'minimum_nights_avg_ntm', 'maximum_nights_avg_ntm']

corr_noches = df_marzo[cols_noches].corr()

plt.figure(figsize=(8, 6))
sns.heatmap(corr_noches, annot=True, cmap="coolwarm", fmt='.2f', vmin=-1, vmax=1)
plt.title("Correlación entre columnas de noches mínimas y máximas")
plt.tight_layout()
plt.show()

pct_igual_min= (df_marzo["minimum_nights"] == df_marzo["minimum_minimum_nights"]).mean()
pct_igual_max= (df_marzo["maximum_nights"] == df_marzo["maximum_maximum_nights"]).mean()
print(f'minimum_nights == minimum_minimum_nights en el {pct_igual_min:.2f}% de filas')
print(f'maximum_nights == maximum_maximum_nights en el {pct_igual_max:.2f}% de filas')
```



```
minimum_nights == minimum_minimum_nights en el 1.00% de filas
maximum_nights == maximum_maximum_nights en el 1.00% de filas
```

```
#decidimos eliminar las variables que muestran la misma informacion
df_marzo=df_marzo.drop(columns=['minimum_minimum_nights', 'maximum_maximum_nights', 'maximum_minimum_nights',
                                'minimum_maximum_nights', 'maximum_nights_avg_ntm', 'minimum_nights_avg_ntm'])
```

```
# numero de baños - extraemos el digito del texto
df_marzo["bathrooms_num"] = df_marzo["bathrooms_text"].str.extract(r"([\d.]+)").astype(float)
df_marzo.loc[df_marzo["bathrooms_text"] == "Half-bath", "bathrooms_num"] = 0.5
#si es compartido (informacion que "bathrooms" no tenia) añadimos a dataset principal
df_marzo["shared_bathroom"] = df_marzo["bathrooms_text"].str.contains("shared", case=False, na=False)
df_marzo[["bathrooms_num", "shared_bathroom"]].head(10)
df_marzo=df_marzo.drop(columns=['bathrooms_text', 'bathrooms'])
```

✓ Grupo de variables calculated_host

```
df_marzo[["calculated_host_listings_count", "host_listings_count"]].corr()
```

	calculated_host_listings_count	host_listings_count
calculated_host_listings_count	1.000000	0.91633
host_listings_count	0.91633	1.00000

```
#se eliminan columnas que redundan informacion de host listings, queda solo la variable alculated_host_listings_count
df_marzo=df_marzo.drop(columns=['calculated_host_listings_count_entire_homes',
                                'calculated_host_listings_count_private_rooms',
                                'calculated_host_listings_count_shared_rooms'])
```

✓ Verificación de propiedades repetidas por anfitrión y ubicación

```
df_marzo["coord"] = df_marzo["latitude"].astype(str) + "_" + df_marzo["longitude"].astype(str)
```

```
# calculamos propiedades distintas por anfitrión (una fila por host_id)
host_propiedades = df_marzo.groupby("host_id")["coord"].nunique().reset_index()
host_propiedades.columns = ["host_id", "n_propiedades_distintas"]
```

✓ Generamos columna 'es_gran_tenedor' para los anfitriones que posean mas de 5 propiedades

```
# Primero, fusionamos el número de propiedades distintas por anfitrión en df_marzo
df_marzo = df_marzo.merge(host_propiedades[["host_id", "n_propiedades_distintas"]], on='host_id', how='left')

# Clasificamos a los anfitriones entre privados y gran tenedor =>5 propiedades
df_marzo["es_gran_tenedor"] = df_marzo["n_propiedades_distintas"] >= 5
df_marzo['es_gran_tenedor'].value_counts() # numero de anuncios

df_marzo=df_marzo.drop(columns=['coord'])
```

✓ Bedrooms y beds

```
df_marzo[["bedrooms", "beds"]].corr()
```

	bedrooms	beds
bedrooms	1.000000	0.842091
beds	0.842091	1.000000

```
df_marzo=df_marzo.drop(columns=['beds'])
```

✓ Host_about - texto libre - biografia de anfitrion

```
df_marzo['tiene_biografia'] = df_marzo["host_about"].notna()
```

```
df_marzo=df_marzo.drop(columns=['host_about'])
```

✓ Licence - dato con contexto regulatorio

```
#clasificamos la variable con mucho texto a una categorica

def clasificar_licencia(val):
    if pd.isna(val):
        return 'Sin dato'
    v = val.upper()
    tiene_numero = bool(re.search(r'(HUTB|HB-|AJ\d|ES[A-Z]{2,4}\d{5,})', v))
    tiene_exento = 'EXEMPT' in v
    if tiene_numero:
        return 'Licencia registrada'
```

```

        if tiene_exento:
            return 'Exento'
        return 'Otro'

df_marzo['licencia_estado'] = (
    df_marzo['license']
    .apply(clasificar_licencia)
    .astype('category')
)

print(df_marzo['licencia_estado'].value_counts())
df_marzo=df_marzo.drop(columns=['license'])

```

```

licencia_estado
Licencia registrada    10858
Sin dato               4646
Exento                 384
Otro                   219
Name: count, dtype: int64

```

Reviews columnas

```

df_marzo=df_marzo.drop(columns=[ 'number_of_reviews_ly', 'review_scores_accuracy',
    'review_scores_cleanliness', 'review_scores_checkin', 'review_scores_communication',
    'review_scores_value'])

```

Variables de ubicacion

```

#definimos un umbral de minimo de anuncios por barrio para agrupar categorias
umbral = 150
conteo_barrios = df_marzo['neighbourhood_cleansed'].value_counts()
#Quedarían 20 categorias en esta variable
df_marzo['neighbourhood_agrupado']= df_marzo['neighbourhood_cleansed'].apply(lambda x: x if conteo_barrios[x] >= umbral else 'Otro')
df_marzo.drop(columns=['neighbourhood_cleansed'], inplace=True)

```

ID Columnas a eliminar

```

df_marzo.drop(columns=['host_profile_id','scrape_id'], inplace=True)

```

Familia variables host

```

df_marzo['antiguedad_como_user'] = df_marzo['hosts_time_as_user_years'] * 12 + df_marzo['hosts_time_as_user_months']
df_marzo['antiguedad_como_host'] = df_marzo['hosts_time_as_host_years'] * 12 + df_marzo['hosts_time_as_host_months']

```

```

df_marzo=df_marzo.drop(columns=['hosts_time_as_host_years', 'hosts_time_as_host_months', 'hosts_time_as_user_years', 'hosts_time_as_user_months'])

```

```

df_marzo=df_marzo.drop(columns=['host_is_superhost'])

```

Familia Reviews

```

#generamos una columna - es True si no hay reviews
df_marzo['sin_reviews'] = df_marzo['reviews_per_month'].isna()

```

```

#rellenamos los nulos - ya que no hay reseñas, no hay puntualización ni frecuencia que rellenar - sería 0
df_marzo[['review_scores_location', 'review_scores_rating', 'reviews_per_month']] = df_marzo[['review_scores_location', 'review_scores_rating', 'reviews_per_month']].fillna(0)

```

```

#host_location - de texto libre y alta cardinalidad, los nulos rellenamos con "desconocido"
df_marzo['host_location'].fillna('Desconocido', inplace=True)

```

```

# Creación de variable host_location_agrupado
df_marzo['host_location_agrupado'] = (
    agrupar_raras(df_marzo['host_location'], umbral_pct=1.0)
    .astype('category')
)

```

```
df_marzo['property_type_agrupado'] = agrupar_raras(df_marzo['property_type'], umbral_pct=1.0).astype('category')
```

```
#eliminamos la variable original y quedamos con agrupada
df_marzo.drop(columns=['host_location', 'property_type'], inplace=True)
```

```
#eliminamos las variables por estar correlacionadas altamente
df_marzo.drop(columns=['availability_30', 'availability_60', 'availability_90'], inplace=True)
```

```
#Eliminamos las dos variables
df_marzo.drop(columns=['estimated_revenue_l365d', 'estimated_occupancy_l365d'], inplace=True)
```

```
#description - convertimos en booleano - tiene o no descripción y imputamos por moda - eliminamos la original
df_marzo['tiene_descripcion'] = df_marzo['description'].notna()
df_marzo = df_marzo.drop(columns=['description'])
```

```
#Variables last_review y first_review - con 4112 nulos cada una - no son datos perdidos, es que simplemente aun no tienen
# las usaremos junto con la variable last_scraped para generar antigüedad de anuncio en días
df_marzo['antigüedad_anuncio'] = (pd.to_datetime(df_marzo['last_scraped']) - df_marzo['first_review']).dt.days
```

```
#eliminamos las variables iniciales, ya no pueden aportar nada más
df_marzo = df_marzo.drop(columns=['last_scraped', 'first_review', 'last_review', 'calendar_last_scraped'])
```

3. Tratamiento de datos nulos

```
#rellenamos los nulos por mediana
df_marzo['antigüedad_anuncio'] = df_marzo['antigüedad_anuncio'].fillna(df_marzo['antigüedad_anuncio'].median())
```

```
#host_name con categoría de desconocidos
df_marzo['host_name'] = df_marzo['host_name'].fillna('Desconocido')
```

```
# Imputación de booleanas por moda
for col in ['host_has_profile_pic', 'host_identity_verified', 'has_availability', 'tiene_descripcion']:
    df_marzo[col] = df_marzo[col].fillna(df_marzo[col].mode()[0])
```

```
#imputación de bedrooms, número de bathrooms y acommodates por Iterative Imputer
cols_iterative = ['bedrooms', 'bathrooms_num', 'accommodates']
imputer = IterativeImputer(max_iter=10, random_state=42)
df_marzo[cols_iterative] = imputer.fit_transform(df_marzo[cols_iterative])
```

```
# Imputación Chain equations: para las otras numéricas
imputer_itImp = skl_imp.IterativeImputer(max_iter=20, random_state=0)
df_marzo_variables_cont = ["host_listings_count", "antigüedad_como_user", "antigüedad_como_host",
                           "maximum_nights", "minimum_nights"]
df_marzo[df_marzo_variables_cont] = skl_imp.IterativeImputer(max_iter=20, random_state=0).fit_transform(df_marzo[df_marzo_variables_cont])
```

```
# price - imputamos por medianas, usando distrito y room_type (SOLO IMPUTAMOS PRICE COMO PRUEBA)
df_marzo['price'] = df_marzo.groupby(['neighbourhood_agrupado', 'room_type'])['price'].transform(lambda x: x.fillna(x.median()))
```

4. Gestión de Outliers

```
#la variable price - pasamos a log (explicar porque convertimos a logaritmo)
```

```
# Convertir Precio a Logaritmo
df_marzo['price_log'] = np.log(df_marzo['price'])
df_marzo = df_marzo.dropna(subset=['price_log'])
print(f"Dataset actualizado: {df_marzo.shape}")
```

Dataset actualizado: (16106, 43)

```
#tratamiento de outliers restantes:
```

```
cols_winsorizar = ['host_listings_count', 'calculated_host_listings_count', 'n_propiedades_distintas',
                  'number_of_reviews', 'number_of_reviews_ltm', 'number_of_reviews_l30d', 'reviews_per_month',
                  'accommodates']
```

```
for col in cols_winsorizar:
```

```

q1, q3 = df_marzo[col].quantile([0.25, 0.75])
iqr = q3 - q1
lim_inf = max(q1 - 1.5 * iqr, df_marzo[col].min()) #no bajamos el minimo real posible
lim_sup = q3 + 1.5 * iqr

df_marzo[col] = df_marzo[col].clip(lower=lim_inf, upper=lim_sup)

```

```

# se aplica cifra maxima a extreme outliers en estas variables
for col in ['maximum_nights']:
    df_marzo.loc[df_marzo[col] > 100_000, col] = 1125

```

5. Relaciones entre variables y analisis estadistico

```

#Estadisticos descriptivos

```

```

numericas = df_marzo.select_dtypes(include=[np.number]).columns.tolist()
numericas = [c for c in numericas if c not in ['id', 'host_id', 'scrape_id']] #excluimos columnas de id

categoricas = df_marzo.select_dtypes(include=['category', 'bool']).columns.tolist()
categoricas = [c for c in categoricas if c not in ['name', 'host_name', 'amenities', 'last_scraped', 'calendar_last_scrape']]

print(f'{len(numericas)} numéricas:', numericas)
print(f'{len(categoricas)} categóricas:', categoricas)

```

```

23 numéricas: ['host_listings_count', 'latitude', 'longitude', 'accommodates', 'bedrooms', 'price', 'minimum_nights', 'maximum_nights', 'host_has_profile_pic', 'host_identity_verified', 'neighbourhood_group_cleansed', 'room_type', 'has_availability', 'last_scraped', 'calendar_last_scrape', 'amenities', 'source', 'name', 'host_name', 'id', 'host_id', 'scrape_id']
14 categóricas: ['room_type', 'neighbourhood_group_cleansed', 'host_identity_verified', 'host_has_profile_pic', 'last_scraped', 'calendar_last_scrape', 'amenities', 'source', 'name', 'host_name', 'id', 'host_id', 'scrape_id']

```

```

#añadimos variables aleatorias para el control
df_marzo['aleatorio1'] = np.random.normal(0, 1, size=len(df_marzo))
df_marzo['aleatorio2'] = np.random.normal(0, 1, size=len(df_marzo))

```

```

y = df_marzo['price_log']

excluir = ['price', 'price_log', 'id', 'host_id',
           'last_scraped', 'source', 'name',
           'amenities', 'host_name']

x = df_marzo.drop(columns=[c for c in excluir if c in df_marzo.columns])

tablaCramer=pd.DataFrame(
    x.apply (lambda col: crammers_v(col, y)),
    columns = ['VCramer']).sort_values(by='VCramer', ascending=False)

```

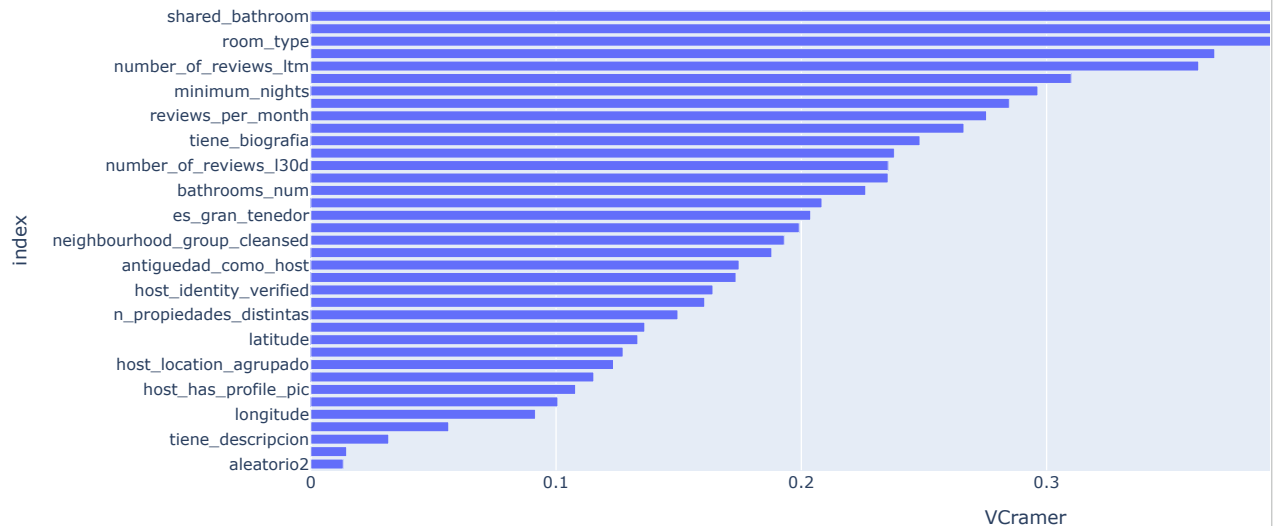
```

import plotly.express as px

# Obtener el gráfico de importancia de las variables frente a la objetivo continua según vcramer
px.bar(tablaCramer,x=tablaCramer.VCramer,title='Relaciones frente al price_log').update_yaxes(categoryorder="total ascending")

```

Relaciones frente al price_log



6. Feature ingenering de variables

```
df_marzo['has_ac'] = df_marzo['amenities'].str.contains('Air conditioning', case=False, na=False)
df_marzo['has_elevator'] = df_marzo['amenities'].str.contains('Elevator', case=False, na=False)
df_marzo['has_workstation'] = df_marzo['amenities'].str.contains('Dedicated workspace', case=False, na=False)
df_marzo['has_dishwasher'] = df_marzo['amenities'].str.contains('Dishwasher', case=False, na=False)
df_marzo['has_balcony'] = df_marzo['amenities'].str.contains('balcony', case=False, na=False)
df_marzo['has_parking'] = df_marzo['amenities'].str.contains('parking', case=False, na=False)
df_marzo['has_pool'] = df_marzo['amenities'].str.contains('Pool', case=False, na=False)
df_marzo['has_self_checkin'] = df_marzo['amenities'].str.contains('Self check-in', case=False, na=False)
#despues eliminamos la variable amenities
df_marzo.drop(columns=['amenities'], inplace=True)
```

```
#creación de nuevas variables de distancia según geo localización
import geopandas as gpd
# Cargar el archivo con las coordenadas ya geocodificadas
geo_output = gpd.read_file("geo_barcelona.geojson")
geo_output = geo_output.to_crs(epsg=4326)

geo_output['xlong'] = geo_output['geometry'].x
geo_output['ylat'] = geo_output['geometry'].y

geo_output[['address', 'ylat', 'xlong']]
```

	address	ylat	xlong	
0	Barcelona-Plaça Catalunya, Carrer de Pelai, 08...	41.385667	2.168373	
1	Barceloneta, Plaça de Pau Vila, 08003, Plaça d...	41.382042	2.185443	
2	Basílica de la Sagrada Família, Carrer de Mall...	41.403505	2.174428	
3	Parc Güell, Barcelona, Catalunya, España	41.414235	2.152458	
4	Spotify Camp Nou, Avinguda de Joan XXIII, 0802...	41.380881	2.122818	

```
#para calcular distancias precisas ente anuncios y los puntos de intreses usamos sistema EPSG:25831
```

```
#convertimos df_marzo a GeoDataFrame con las coordenada originales
```

```
gdf_marzo = gpd.GeoDataFrame(
    df_marzo,
    geometry=gpd.points_from_xy(df_marzo['longitude'], df_marzo['latitude']),
    crs='EPSG:4326'
)
```

```
gdf_marzo = gdf_marzo.to_crs(epsg=25831)
geo_output_utm = geo_output.to_crs(epsg=25831)
```

```

geo_output_utm.index = ['dist_center_km', 'dist_beach_km', 'dist_sagrada_familia_km', 'dist_park_guell_km', 'dist_camp_nou']

for nombre, punto in geo_output_utm['geometry'].items():
    gdf_marzo[nombre] = gdf_marzo.geometry.distance(punto)/1000 # a km

df_marzo = pd.DataFrame(gdf_marzo.drop(columns='geometry'))
df_marzo[list(geo_output_utm.index)].describe()

#eliminamos latitud y longitud
df_marzo.drop(columns=['latitude', 'longitude'], inplace=True)

```

```
df_marzo = df_marzo.drop(columns = ['price', 'host_id', 'source', 'name', 'host_name'])
```

```

columnas_eliminar = ['neighbourhood_group_cleansed', 'number_of_reviews', 'number_of_reviews_l30d', 'reviews_per_month', 'reviews_per_month_l30d']
df_marzo = df_marzo.drop(columns=[c for c in columnas_eliminar if c in df_marzo.columns])

```

```

cols_categoricas = df_marzo.select_dtypes(include=['category', 'boolean', 'bool']).columns.tolist()
print(cols_categoricas)
#Convertimos en dummy y lo junta con las otras variables
df_marzo = pd.get_dummies(df_marzo, columns=cols_categoricas, drop_first=True)

```

```
['host_has_profile_pic', 'host_identity_verified', 'room_type', 'has_availability', 'shared_bathroom', 'es_gran_tenedor', 'total_reviews']
```

```

df_marzo.columns = (
    df_marzo.columns
    .str.replace(' ', '_', regex=False) #
    .str.replace("'", '_', regex=False) #
    .str.replace('-', '_', regex=False) #
    .str.replace(',', '_', regex=False) #
    .str.replace(r'[^\\w]', '', regex=True) #
    .str.replace('__+', '_', regex=True)
    .str.strip('_') #
)

```

```

lista_bool = df_marzo.select_dtypes(include=['bool', 'boolean']).columns
df_marzo[lista_bool] = df_marzo[lista_bool].astype('int')

```

8. Particion training-test

```

from sklearn.model_selection import train_test_split #dividimos en train/test

X_train, X_test, y_train, y_test = train_test_split(df_marzo.drop(['price_log', 'id'], axis=1),
                                                    df_marzo.price_log,
                                                    test_size=0.2,
                                                    random_state=50)

```

```

data_train= X_train.join(y_train)
data_train.head()

```

[Mostrar salida oculta](#)

Modelo OLS

```

#MODELO COMPLETO
from statsmodels.formula.api import ols

form_completo = ols_formula(data_train, 'price_log')
modeloC = ols(form_completo, data=data_train).fit()
print(modeloC.summary())

```

[Mostrar salida oculta](#)

```

# Función para facilitar el cálculo del vif
from statsmodels.stats.outliers_influence import variance_inflation_factor
# VIF modelo completo
tabla_vif = vif_modelo(form_completo,data_train)

tabla_vif = tabla_vif[tabla_vif["VIF"] < 5]
cols_vif = tabla_vif["feature"].tolist()

```

```
cols_vif
form_vif='price_log ~'+ ' '.join(cols_vif)
```

```
# Modelo sin aleatorios
form2=ols_formula(data_train.drop(['aleatorio1','aleatorio2'], axis=1),'price_log')

# modelo sin variables no significativas y aleatorios
significativas= ['neighbourhood_agrupado_el_Camp_d_en_Grassot_i_Gràcia_Nova','neighbourhood_agrupado_Sant_Antoni','maximum_
neighbourhood_agrupado_la_Vila_de_Gràcia','has_pool_True','dist_camp_nou_km','tiene_biografia_True','neighbourhood_agrupa
review_scores_rating','dist_park_guell_km','licencia_estado_Sin_datos','property_type_agrupado_Private_room_in_hostel','ha
bedrooms','number_of_reviews_ltm','calculated_host_listings_count','bathrooms_num','antiguedad_como_host','minimum_nights
room_type_Shared_room','shared_bathroom_True','es_gran_tenedor_True','licencia_estado_Licencia_registrada','neighbourhood
neighbourhood_agrupado_el_Fort_Pienc','neighbourhood_agrupado_el_Raval','neighbourhood_agrupado_la_Barceloneta','neighbour
sin_reviews_True','host_location_agrupado_Desconocido','property_type_agrupado_Room_in_hotel','tiene_descripcion_True','ha
has_parking_True','has_self_checkin_True']

# Incluir la variable objetivo 'price_log' en el DataFrame antes de pasarlo a ols_formula
columns_for_form3 = significativas + ['price_log']
form3 =ols_formula(data_train[columns_for_form3],'price_log')
form3

# Creamos lista de fórmulas
list_form = [form_completo,form2, form3, form_vif]
list_form

# Aplicamos a toda la lista la función creada (devuelve un dataframe pero está transpuesto)
list_res = pd.DataFrame(map(lambda x: cross_val_lin(x, data_train, seed=2022),list_form))

# Transponer dataframe y pasar de wide a long (creando un factor variable con el nombre de cada fórmula de la lista[0,1,2,3])
results = list_res.T.melt()
results.columns = ['Modelo','R2']
results.head()
```

Modelo: price_log ~ accommodates + bedrooms + minimum_nights + maximum_nights + number_of_reviews_ltm + availability_eoy + r
 Coeficiente de determinación R2: 0.687 (0.014)
 Modelo: price_log ~ accommodates + bedrooms + minimum_nights + maximum_nights + number_of_reviews_ltm + availability_eoy + r
 Coeficiente de determinación R2: 0.687 (0.014)
 Modelo: price_log ~ neighbourhood_agrupado_el_Camp_d_en_Grassot_i_Gràcia_Nova + neighbourhood_agrupado_Sant_Antoni + maximum
 Coeficiente de determinación R2: 0.685 (0.014)
 Modelo: price_log ~neighbourhood_agrupado_la_Nova_Esquerre+neighbourhood_agrupado_el_Poblenou+neighbourhood_agrupado_la_Vila_de_Gràcia
 Coeficiente de determinación R2: 0.638 (0.016)

	Modelo	R2	
0	0	0.653895	
1	0	0.674169	
2	0	0.695266	
3	0	0.681680	
4	0	0.730219	

Pasos siguientes: [Generar código con results](#) [New interactive sheet](#)

```
# Aplicamos a fórmula de modelo completo

# Generamos las matrices de diseño según la fórmula de modelo completo
y, X_patsy = patsy.dmatrices(form_completo, data_train, return_type='dataframe')
```

```
from sklearn import linear_model

reg = linear_model.LassoLarsIC(criterion='bic')
reg.fit(X_patsy, y)
```

▼ LassoLarsIC ⓘ ?
 LassoLarsIC(criterion='bic')

```
selec_feats = X_patsy[X_patsy.columns[(reg.coef_ != 0).ravel().tolist()]]
selec_feats.columns.tolist()
```

[Mostrar salida oculta](#)

Validacion cruzada

```
import seaborn as sns
from sklearn.model_selection import cross_val_score
```

```
from sklearn.model_selection import RepeatedKFold
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
list_dif_modelos = ['selec_feats']
# Aplicamos a toda la lista la función creada (devuelve un dataframe pero está transpuesto)
list_res = pd.DataFrame(map(lambda x: cross_val(data_train.drop(['price_log'], axis=1), data_train, data_train['price_log']),
                             list_dif_modelos))

# Transponer dataframe y pasar de wide a long (creando un factor variable con el nombre de cada fórmula de la lista)
results = list_res.T.melt()
results.columns = ['Modelo', 'R2']
results.head()
```

Coeficiente de determinación R2: 0.687 (0.014)

	Modelo	R2	
0	0	0.653895	
1	0	0.674169	
2	0	0.695266	
3	0	0.681680	
4	0	0.730219	

Pasos siguientes:

[Generar código con results](#)

[New interactive sheet](#)

10. Probamos diferentes algoritmos modernos

```
data_imput = data_train[columns_for_form3] #sigue la forma de form3
```

```
X = data_imput.drop(columns=["price_log"])
y = data_imput["price_log"]
```

```
modelos = {
    'Linear Regression': LinearRegression(),
    'Ridge': Ridge(random_state=123),
    'Lasso': Lasso(random_state=123),
    'Decision Tree': DecisionTreeRegressor(random_state=123),
    'Random Forest': RandomForestRegressor(random_state=123),
    'Gradient Boosting': GradientBoostingRegressor(random_state=123),
    'XGBoost': XGBRegressor(random_state=123),
    'KNN': KNeighborsRegressor(),
}

cv = RepeatedKFold(n_splits=5, n_repeats=3, random_state=123)

resultados = []

for nombre, modelo in modelos.items():
    scores = cross_val_score(modelo, X, y, scoring='r2', cv=cv, n_jobs=1)
    print(f"{nombre}: R2 = {scores.mean():.3f} ({scores.std():.3f})")
    resultados.append([nombre, np.mean(scores), np.std(scores)])
```

```
Linear Regression: R2 = 0.686 (0.014)
Ridge: R2 = 0.686 (0.014)
Lasso: R2 = 0.230 (0.009)
Decision Tree: R2 = 0.672 (0.010)
Random Forest: R2 = 0.830 (0.007)
Gradient Boosting: R2 = 0.791 (0.008)
XGBoost: R2 = 0.825 (0.009)
KNN: R2 = 0.656 (0.014)
```

```
tabla_resultados = pd.DataFrame(resultados, columns=['Modelo', 'R2_medio', 'R2_std'])
tabla_resultados = tabla_resultados.sort_values('R2_medio', ascending=False)
tabla_resultados
```

	Modelo	R2_medio	R2_std	
4	Random Forest	0.829882	0.006672	 
6	XGBoost	0.824963	0.008524	
5	Gradient Boosting	0.790932	0.008340	
1	Ridge	0.685618	0.014498	
0	Linear Regression	0.685609	0.014488	
3	Decision Tree	0.671567	0.010172	
7	KNN	0.656390	0.013731	
2	Lasso	0.230446	0.008781	

Pasos siguientes:

[Generar código con tabla_resultados](#)

[New interactive sheet](#)

11. Ajuste de hiperparametros - XGBoost

```
param_dist_v2 = {
    'n_estimators': [200, 300, 500, 700, 900], #arboles se ontstruyen secuencialmente
    'max_depth': [4, 5, 6, 8, 10, 12],
    'learning_rate': [0.01, 0.03, 0.05, 0.1],
    'subsample': [0.7, 0.8, 0.9, 1.0],
    'colsample_bytree': [0.8, 0.9, 1.0],
    'min_child_weight': [1, 3, 5],
}

xgb_v2 = XGBRegressor(random_state=123, n_jobs=-1)

random_search_v2 = RandomizedSearchCV(
    xgb_v2,
    param_distributions=param_dist_v2,
    n_iter=30,
    scoring='r2',
    cv=5,
    random_state=123,
    n_jobs=1,
    verbose=1
)

random_search_v2.fit(X, y)

print("Mejores hiperparámetros (ampliado):", random_search_v2.best_params_)
print("Mejor R2 (CV) ampliado:", random_search_v2.best_score_)

Fitting 5 folds for each of 30 candidates, totalling 150 fits
Mejores hiperparámetros (ampliado): {'subsample': 0.9, 'n_estimators': 700, 'min_child_weight': 3, 'max_depth': 8, 'learning
Mejor R2 (CV) ampliado: 0.8407193193910694
```

12. Ajuste hiperparametros - Random Forest

```
param_dist_rf_v2 = {
    'n_estimators': [100, 200, 300, 500], #arboles se construyen de forma independiente y se promedian
    'max_depth': [10, 15, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2', 0.5, 0.8],
}

rf_v2 = RandomForestRegressor(random_state=123, n_jobs=-1)

random_search_rf_v2 = RandomizedSearchCV(
    rf_v2,
    param_distributions=param_dist_rf_v2,
    n_iter=30,
    scoring='r2',
    cv=5,
    random_state=123,
    n_jobs=1,
    verbose=1
)
```

```
random_search_rf_v2.fit(X, y)
```

```
print("Mejores hiperparámetros RF (ampliado):", random_search_rf_v2.best_params_)  
print("Mejor R2 (CV) RF ampliado:", random_search_rf_v2.best_score_)
```

Fitting 5 folds for each of 30 candidates, totalling 150 fits

Mejores hiperparámetros RF (ampliado): {'n_estimators': 200, 'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features':
Mejor R2 (CV) RF ampliado: 0.8348056098893514

✓ 13. Predicciones con Xtest

```
mejor_modelo = random_search_v2.best_estimator_
```

```
X_test_final = X_test[significativas]  
pred_test = mejor_modelo.predict(X_test_final)
```

```
r2_test = r2_score(y_test, pred_test)  
rmse_test = np.sqrt(mean_squared_error(y_test, pred_test))
```

```
print(f"R² test: {r2_test:.4f}")  
print(f"RMSE test: {rmse_test:.4f}")
```

R² test: 0.8471
RMSE test: 0.3328

Se observa que R2 para el modelo xgboost en este caso es de 0.847. Se asimila al

- ✓ R2 del modelo que se entrena sin imputar precio. Se decide no imputar precio antes de entrenar el modelo.

Empieza a programar o a [crear código](#) con IA.